API Flask/FastAPI

m2iformation.fr

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

API Flask/FastAPI

Utopios® Tous droits réservés

Module 4 - Patterns Avancés + Architecture

Architecture Scalable et Patterns Professionnels

MVC, Service Layer, Repository, Versioning, CQRS et sécurité avancée

Patterns | Architecture | Sécurité Avancée

Utopios® Tous droits réservés

MVC Pattern (Model-View-Controller)

Architecture fondamentale

MVC sépare l'application en trois couches:

- **Model:** Représente les données (Pydantic, SQLAlchemy)
- **View:** Présentation des données (JSON, HTML)
- **Controller:** Logique métier (routes, handlers)

Utopios® Tous droits réservés

MVC Pattern (Model-View-Controller)

```
# Structure de projet MVC
app/
├── models/          # Pydantic et ORM
│   └── user.py
├── controllers/     # Routes et handlers
│   └── user_controller.py
├── views/          # Sérialisation
│   └── user_view.py
└── main.py
```

Avantage: Séparation des responsabilités claire

Utopios® Tous droits réservés

Service Layer Pattern

Couche métier réutilisable

Isolez la logique métier dans une couche Service:

```python
class UserService:
    """Logique métier utilisateurs"""
    def __init__(self, repository: UserRepository):
        self.repository = repository

    async def create_user(self, user_data: UserCreate) -> User:
        # Validation métier
        if await self.repository.find_by_email(user_data.email):
            raise ValueError("Email déjà utilisé")

        # Hachage du mot de passe
        hashed_password = hash_password(user_data.password)

        # Créer l'utilisateur
        user = await self.repository.create({
            **user_data.dict(),
            "password": hashed_password
        })

        # Envoyer un email de bienvenue
        await send_welcome_email(user.email)

        return user

    async def get_user(self, user_id: int) -> User:
        return await self.repository.find_by_id(user_id)
```

Utopios® Tous droits réservés

Repository Pattern

Abstraction de l'accès aux données

Le Repository centralise l'accès aux données:

```python
from abc import ABC, abstractmethod

class UserRepository(ABC):
    """Interface pour accès utilisateurs"""

    @abstractmethod
    async def find_by_id(self, user_id: int) -> User:
        pass

    @abstractmethod
    async def find_by_email(self, email: str) -> User:
        pass

    @abstractmethod
    async def create(self, data: dict) -> User:
        pass

    @abstractmethod
    async def update(self, user_id: int, data: dict) -> User:
        pass

    @abstractmethod
    async def delete(self, user_id: int) -> bool:
        pass

class SQLAlchemyUserRepository(UserRepository):
    def __init__(self, db_session):
        self.db = db_session

    async def find_by_id(self, user_id: int):
        return await self.db.query(User).filter(User.id == user_id).first()
```

Utopios® Tous droits réservés

Unit of Work Pattern

Transactions et commits atomiques

Contrôlez les transactions de base de données:

```
class UnitOfWork:
    """Gère les transactions de la BD"""
    def __init__(self, db_session):
        self.db = db_session
        self.users = UserRepository(db_session)
        self.orders = OrderRepository(db_session)

    async def __aenter__(self):
        return self

    async def __aexit__(self, exc_type, exc, tb):
        if exc_type:
            await self.db.rollback()
        else:
            await self.db.commit()
        await self.db.close()

# Utilisation
async def create_user_with_order(user_data, order_data):
    async with UnitOfWork(db) as uow:
        user = await uow.users.create(user_data)
        order = await uow.orders.create({**order_data, "user_id": user.id})
        # Tout est commité ou tout est rollbacker en cas d'erreur
        return user, order
```

Utopios® Tous droits réservés

Dependency Injection Container

Gérer les dépendances

Utilisez un conteneur DI pour injecter les dépendances:

```python
from dependency_injector import containers, providers

class Container(containers.DeclarativeContainer):
    """DI Container"""
    config = providers.Configuration()

    db = providers.Singleton(
        Database,
        url=config.database.url
    )

    user_repository = providers.Factory(
        UserRepository,
        db=db
    )

    user_service = providers.Factory(
        UserService,
        repository=user_repository
    )

# Utilisation dans FastAPI
container = Container()

@app.post("/users")
async def create_user(
    user: UserCreate,
    service: UserService = Depends(container.user_service)
):
    return await service.create_user(user)
```

Utopios® Tous droits réservés

SQLAlchemy ORM

Mapper objet-relationnel

Utilisez SQLAlchemy pour l'ORM:

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey
from sqlalchemy.orm import declarative_base, relationship
from datetime import datetime

Base = declarative_base()

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    email = Column(String(100), unique=True, nullable=False)
    password = Column(String(255), nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Relationship One-to-Many
    orders = relationship("Order", back_populates="user")

class Order(Base):
    __tablename__ = "orders"

    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    total = Column(Float, nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Reverse relationship
    user = relationship("User", back_populates="orders")
```

Utopios® Tous droits réservés

Relationships SQLAlchemy

One-to-Many et Many-to-Many

Modélisez les relations complexes:

```
# One-to-Many: Un utilisateur a plusieurs commandes
class User(Base):
    orders = relationship("Order", back_populates="user")

class Order(Base):
    user_id = Column(Integer, ForeignKey("users.id"))
    user = relationship("User", back_populates="orders")

# Many-to-Many: Des étudiants suivent plusieurs cours
from sqlalchemy import Table

course_student = Table(
    'course_student',
    Base.metadata,
    Column('course_id', Integer, ForeignKey('courses.id')),
    Column('student_id', Integer, ForeignKey('students.id'))
)

class Course(Base):
    students = relationship("Student", secondary=course_student, back_populates="courses")

class Student(Base):
    courses = relationship("Course", secondary=course_student, back_populates="students")
```

Utopios® Tous droits réservés

Migrations avec Alembic

Versionner votre schéma BD

Utilisez Alembic pour les migrations:

```
pip install alembic

# Initialiser Alembic
alembic init alembic

# Créer une migration
alembic revision --autogenerate -m "Add user table"

# Appliquer les migrations
alembic upgrade head

# Revenir en arrière
alembic downgrade -1
```

Utopios® Tous droits réservés

Migrations avec Alembic

Fichier de migration:

```
def upgrade():
    op.create_table(
        'users',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('name', sa.String(100), nullable=False),
        sa.Column('email', sa.String(100), unique=True),
        sa.PrimaryKeyConstraint('id')
    )

def downgrade():
    op.drop_table('users')
```

Utopios® Tous droits réservés

Connection Pooling

Optimiser les connexions BD

Utilisez le pool de connexions:

```
from sqlalchemy.pool import QueuePool
from sqlalchemy import create_engine

engine = create_engine(
    "postgresql://user:pass@localhost/db",
    poolclass=QueuePool,
    pool_size=20,              # Connexions actives
    max_overflow=40,           # Connexions en réserve
    pool_timeout=30,           # Timeout en secondes
    pool_recycle=3600,         # Recréer tous les x secondes
    pool_pre_ping=True         # Vérifier la connexion avant usage
)

# Async version
from sqlalchemy.ext.asyncio import create_async_engine

async_engine = create_async_engine(
    "postgresql+asyncpg://user:pass@localhost/db",
    echo=False,
    pool_size=20,
    max_overflow=40
)
```

Performance: Pool bien configuré = moins de latence BD!

Utopios® Tous droits réservés

API Versioning

Gérer les versions d'API

Implémentez le versioning d'API:

```
# Strategy 1: URL path versioning
@app.get("/v1/users")
async def get_users_v1():
    return {"version": "1.0", "users": []}

@app.get("/v2/users")
async def get_users_v2():
    return {"version": "2.0", "users": [], "total": 0}

# Strategy 2: Header versioning
@app.get("/users", headers={"API-Version": "1.0"})
async def get_users(api_version: str = Header()):
    if api_version == "1.0":
        return {"version": "1.0", "users": []}
    elif api_version == "2.0":
        return {"version": "2.0", "users": [], "total": 0}

# Strategy 3: Query parameter versioning
@app.get("/users")
async def get_users(version: str = Query("1.0")):
    if version == "1.0":
        return {"version": "1.0", "users": []}
    return {"version": "2.0", "users": [], "total": 0}
```

Meilleure pratique: URL path versioning (plus simple)

Utopios® Tous droits réservés

Pagination, Filtering et Sorting

Requêtes complexes et scalables

Implémentez pagination, filtrage et tri:

```python
@app.get("/users")
async def list_users(
    skip: int = Query(0, ge=0),
    limit: int = Query(10, ge=1, le=100),
    sort_by: str = Query("created_at"),
    order: str = Query("desc", regex="^(asc|desc)$"),
    search: str = Query(None)
):
    query = db.query(User)

    # Filtrage
    if search:
        query = query.filter(User.name.ilike(f"%{search}%"))

    # Tri
    sort_column = getattr(User, sort_by)
    if order == "desc":
        query = query.order_by(sort_column.desc())
    else:
        query = query.order_by(sort_column.asc())

    # Pagination
    total = query.count()
    users = query.offset(skip).limit(limit).all()

    return {
        "data": users,
        "total": total,
        "skip": skip,
        "limit": limit
    }
```

Utopios® Tous droits réservés

Rate Limiting Avancé

Protéger contre les abus

Implémentez le rate limiting sophistiqué:

```python
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

# Rate limit global
@app.get("/users")
@limiter.limit("100/minute")
async def get_users(request: Request):
    return {}

# Rate limit par utilisateur
@app.post("/users")
@limiter.limit("10/hour")
async def create_user(request: Request, user: UserCreate):
    return {}

# Rate limit custom
@app.get("/api/expensive")
@limiter.limit("5/minute;100/hour")
async def expensive_operation(request: Request):
    return {"result": "expensive computation"}
```

Attention: Implémentez également côté serveur (Redis) en production!

Utopios® Tous droits réservés

Webhooks

Notifier les clients d'événements

Implémentez les webhooks pour les notifications:

```python
class Webhook(BaseModel):
    url: str
    events: list[str]  # ["user.created", "order.completed"]
    active: bool = True

webhook_repository = {}

@app.post("/webhooks")
async def register_webhook(webhook: Webhook):
    webhook_id = str(uuid4())
    webhook_repository[webhook_id] = webhook
    return {"id": webhook_id, "webhook": webhook}

async def trigger_webhook(event: str, data: dict):
    """Déclenche les webhooks pour un événement"""
    for webhook_id, webhook in webhook_repository.items():
        if webhook.active and event in webhook.events:
            try:
                async with httpx.AsyncClient() as client:
                    await client.post(webhook.url, json={
                        "event": event,
                        "data": data,
                        "timestamp": datetime.utcnow().isoformat()
                    })
            except Exception as e:
                logger.error(f"Webhook failed: {e}")

# Utilisation
@app.post("/users")
async def create_user(user: UserCreate):
    new_user = await service.create_user(user)
    await trigger_webhook("user.created", new_user.dict())
    return new_user
```

Utopios® Tous droits réservés

OAuth2 et JWT Complet

Authentification sécurisée

Implémentez OAuth2 avec JWT:

```
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
import jwt
from datetime import datetime, timedelta

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

def create_access_token(data: dict, expires_delta: timedelta = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(hours=24)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm="HS256")
    return encoded_jwt

async def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        user_id: int = payload.get("sub")
        if user_id is None:
            raise HTTPException(status_code=401)
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401)
    return await UserService().get_user(user_id)

@app.post("/token")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = await authenticate_user(form_data.username, form_data.password)
    if not user:
        raise HTTPException(status_code=401)
    access_token = create_access_token({"sub": user.id})
    return {"access_token": access_token, "token_type": "bearer"}
```

Utopios® Tous droits réservés

RBAC (Role-Based Access Control)

Contrôle d'accès granulaire

Implémentez le RBAC:

```python
from enum import Enum

class Role(str, Enum):
    admin = "admin"
    formateur = "formateur"
    apprenant = "apprenant"

class User(BaseModel):
    id: int
    email: str
    role: Role

async def get_current_user(token: str = Depends(oauth2_scheme)):
    payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
    user_id = payload.get("sub")
    return await UserService().get_user(user_id)

def require_role(*roles: Role):
    async def check_role(current_user: User = Depends(get_current_user)):
        if current_user.role not in roles:
            raise HTTPException(status_code=403, detail="Not authorized")
        return current_user
    return check_role

# Utilisation
@app.delete("/users/{user_id}")
async def delete_user(
    user_id: int,
    current_user: User = Depends(require_role(Role.admin))
):
    return await UserService().delete_user(user_id)
```

Utopios® Tous droits réservés

Encryption et Hashing

Sécuriser les données sensibles

Cryptez et hashez les données:

```python
from cryptography.fernet import Fernet
from passlib.context import CryptContext
import hashlib

# Password hashing
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

# Data encryption
cipher_suite = Fernet(ENCRYPTION_KEY)

def encrypt_data(data: str) -> str:
    return cipher_suite.encrypt(data.encode()).decode()

def decrypt_data(encrypted: str) -> str:
    return cipher_suite.decrypt(encrypted.encode()).decode()

# Utilisation
hashed_pwd = hash_password("mypassword123")
is_valid = verify_password("mypassword123", hashed_pwd)

encrypted_ssn = encrypt_data("123-45-6789")
decrypted_ssn = decrypt_data(encrypted_ssn)
```

Utopios® Tous droits réservés

Secret Management

Gérer les secrets en sécurité

Gérez les secrets sans les exposer:

```python
import os
from dotenv import load_dotenv

load_dotenv()

class Settings(BaseSettings):
    """Configuration avec variables d'environnement"""
    secret_key: str = os.getenv("SECRET_KEY")
    database_url: str = os.getenv("DATABASE_URL")
    api_key: str = os.getenv("API_KEY")
    jwt_algorithm: str = "HS256"

    class Config:
        case_sensitive = True

settings = Settings()

# Fichier .env (JAMAIS en git!)
# SECRET_KEY=super-secret-key-123
# DATABASE_URL=postgresql://user:pass@localhost/db
# API_KEY=sk-1234567890abcdef
```

Utopios® Tous droits réservés

Secret Management

En production, utilisez un gestionnaire de secrets:

- **AWS Secrets Manager:** Gestion centralisée AWS
- **HashiCorp Vault:** Solution open-source
- **Azure Key Vault:** Solution Microsoft

Utopios® Tous droits réservés

Audit Logging

Tracer les actions sensibles

Loggez toutes les actions sensibles:

```python
from datetime import datetime

class AuditLog(BaseModel):
    user_id: int
    action: str
    resource: str
    resource_id: int
    changes: dict
    timestamp: datetime

class AuditService:
    async def log_action(self, user_id: int, action: str, resource: str, resource_id: int, changes: dict = None):
        audit = AuditLog(
            user_id=user_id,
            action=action,
            resource=resource,
            resource_id=resource_id,
            changes=changes or {},
            timestamp=datetime.utcnow()
        )
        await db.save(audit)

# Utilisation
@app.delete("/users/{user_id}")
async def delete_user(user_id: int, current_user: User = Depends(get_current_user)):
    user = await UserService().get_user(user_id)
    await AuditService().log_action(
        current_user.id,
        "DELETE",
        "User",
        user_id,
        {"name": user.name, "email": user.email}
    )
    return await UserService().delete_user(user_id)
```

Utopios® Tous droits réservés

Security Headers

Protéger contre les attaques web

Ajoutez les headers de sécurité:

```python
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware import Middleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://frontend.example.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.middleware("http")
async def add_security_headers(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["X-XSS-Protection"] = "1; mode=block"
    response.headers["Strict-Transport-Security"] = "max-age=31536000; includeSubDomains"
    response.headers["Content-Security-Policy"] = "default-src 'self'"
    return response
```

Headers essentiels: CORS, X-Content-Type-Options, X-Frame-Options, CSP

Utopios® Tous droits réservés

Architecture Finale - 3-Tier

Structure complète et scalable

Voici la structure complète:

```
app/
├── main.py                 # Application FastAPI
├── config.py              # Configuration
├── models/                # Pydantic + SQLAlchemy
│   ├── user.py
│   ├── order.py
├── repositories/          # Accès aux données
│   ├── user_repo.py
│   ├── order_repo.py
├── services/             # Logique métier
│   ├── user_service.py
│   ├── order_service.py
├── api/                  # Routes
│   ├── v1/
│   │   ├── users.py
│   │   ├── orders.py
│   ├── v2/
├── middlewares/          # Intercepteurs
├── security/             # Auth, JWT, RBAC
├── utils/               # Utilitaires
├── tests/               # Tests
└── alembic/            # Migrations BD
```

Scalabilité: Cette architecture supporte 1M+ utilisateurs!

Utopios® Tous droits réservés

Concepts Avancés Summary 1/2

- **MVC Pattern:** Séparation Model-View-Controller
- **Service Layer:** Logique métier centralisée
- **Repository Pattern:** Abstraction accès données
- **Unit of Work:** Transactions atomiques
- **SQLAlchemy ORM:** Mapper objet-relationnel
- **API Versioning:** Gestion des versions

Utopios® Tous droits réservés

Concepts Avancés Summary 2/2

- **Pagination & Filtering:** Requêtes optimisées
- **OAuth2 + JWT:** Authentification sécurisée
- **RBAC:** Contrôle d'accès basé rôles
- **Encryption & Hashing:** Données sécurisées
- **Audit Logging:** Traçabilité complète
- **Security Headers:** Protection web

Utopios® Tous droits réservés

Prochaines Étapes

- **Microservices:** Architecture distribuée avec gRPC
- **Event-Driven:** Kafka, RabbitMQ, Event Sourcing
- **GraphQL:** Alternative à REST
- **Machine Learning:** Intégrer des modèles ML
- **Real-time:** WebSockets, Server-Sent Events
- **Kubernetes:** Orchestration des conteneurs
- **DevOps:** CI/CD, Infrastructure as Code

Ressources: FastAPI docs, SQLAlchemy docs, Pytest docs

Utopios® Tous droits réservés

Conclusion

Vous savez maintenant:

- Construire des APIs REST scalables avec Flask et FastAPI
- Valider les données avec Pydantic
- Implémenter les patterns professionnels
- Sécuriser vos applications avec JWT, OAuth2, RBAC
- Tester complètement avec Pytest
- Documenter automatiquement avec OpenAPI
- Monitorer et logger en production
- Déployer avec Docker et Kubernetes

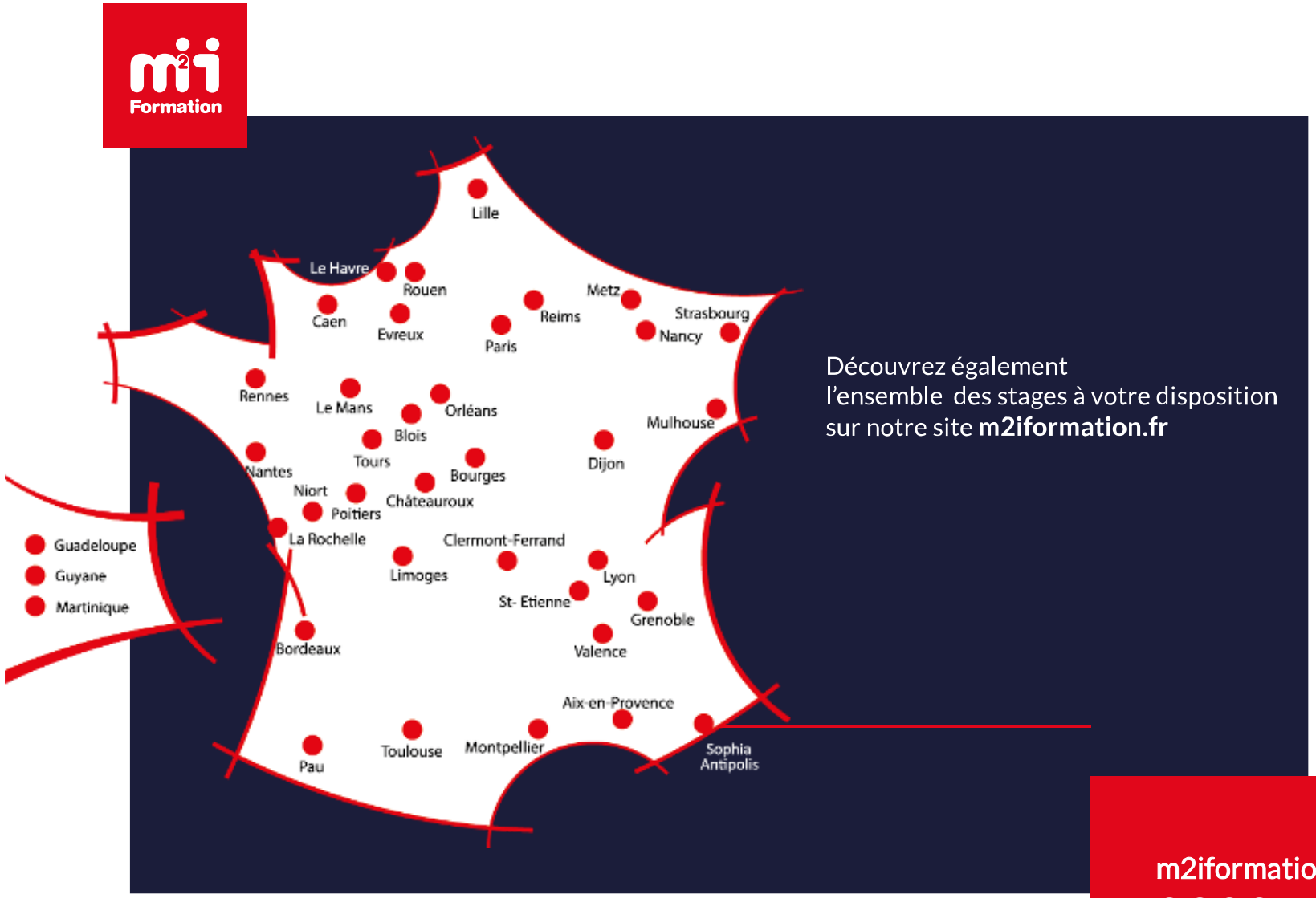
Utopios® Tous droits réservés

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Merci pour votre attention

Des questions ?

Utopios® Tous droits réservés



Découvrez également
l'ensemble des stages à votre disposition
sur notre site **m2information.fr**

m2iformation.fr